



AQB8: Energy-Efficient Ray Tracing Accelerator through Multi-Level Quantization

Yen-Chieh Huang, Chen-Pin Yang, Tsung Tai Yeh

Department of Computer Science

National Yang Ming Chiao Tung University, Taiwan



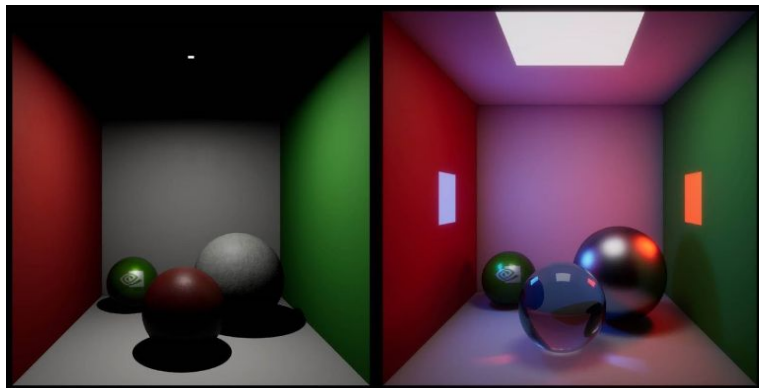
Outline

- Introduction
- Challenge
- Accelerator for
Quantized BVH with
8-bit Traversal (AQB8)
- Methodology
- Evaluation Results
- Conclusion



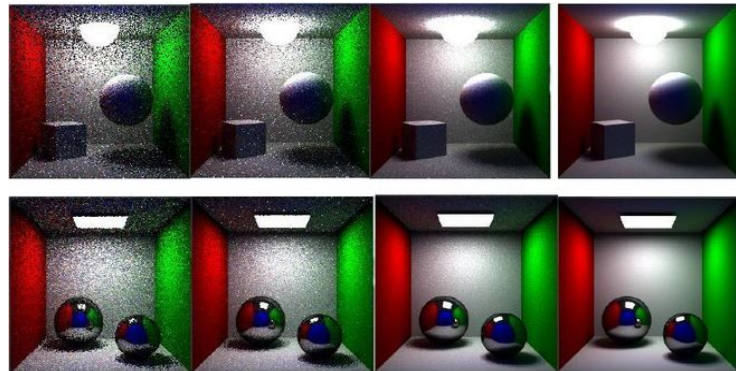
Introduction to Ray Tracing (RT)

- RT creates **high-fidelity graphics rendering**.
- The quality of RT is proportional to the number of rays
 - High computational cost for ray tracing
 - **HW accelerator is needed.**



without RT

with RT



rays per pixel:

1

4

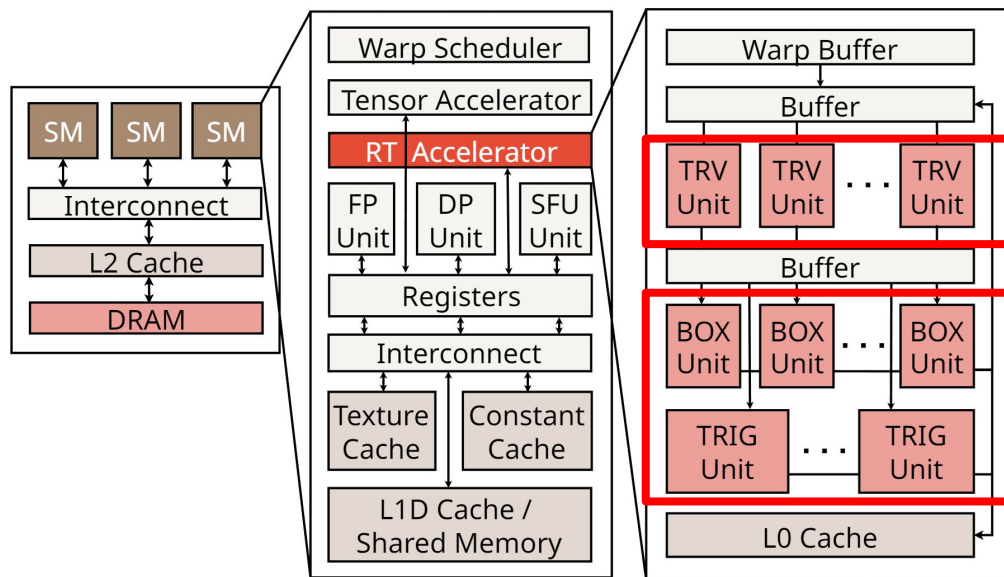
64

1024



Emerging RT Accelerator on GPUs

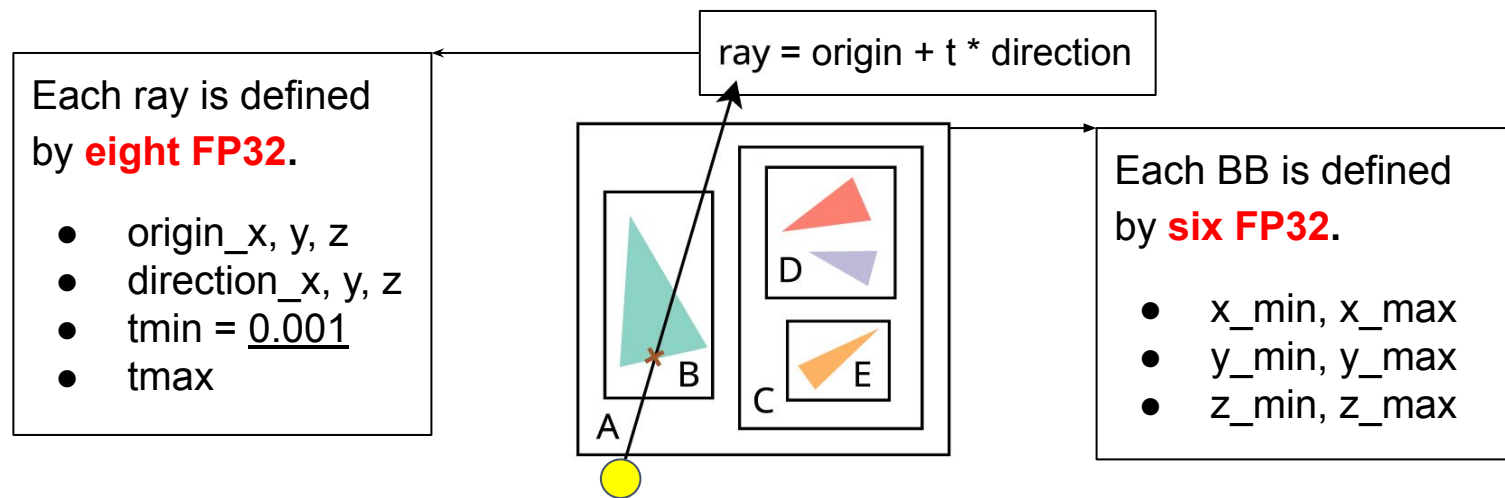
- Modern GPUs (NVIDIA, AMD, Intel) include RT hardware accelerator to achieve real-time graphics rendering.





RT Acceleration through BVH

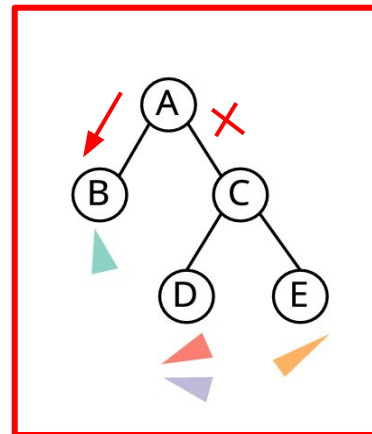
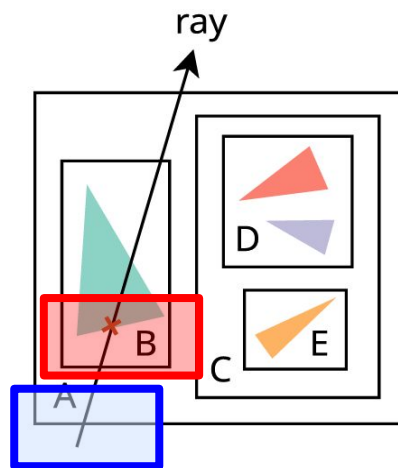
- **BVH (Bounding Volume Hierarchy)** groups **triangles** of an image into **bounding boxes (BBs)**.
- **Rays only test BBs** → avoid RT examines through all triangles.





BVH Traversal Basics

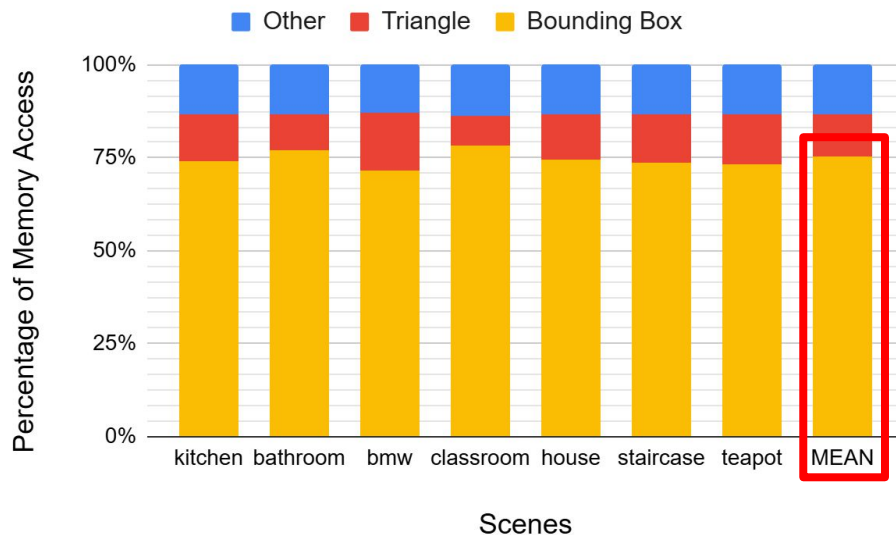
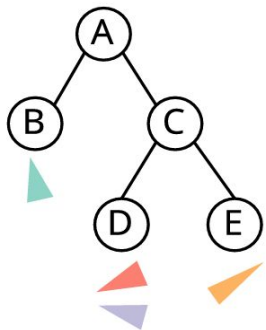
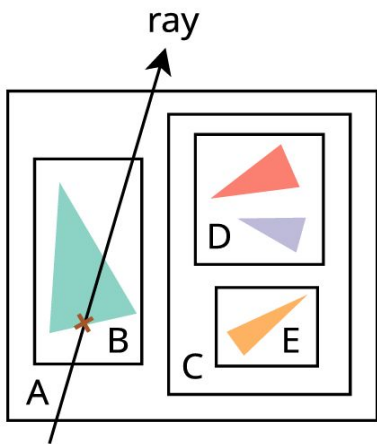
- **BVH Structure:** BBs with triangles at the leaf nodes
Each node of BVH tree represents a BB or a triangle
- **Traversal Process:**
 - Ray starts at the root BB.
 - Visits child BBs if it intersects.
 - Ray-Box Intersection Test
 - Stops at leaf nodes.
 - Ray-Triangle Intersection Test





BVH Tree Traversal Create High DRAM Traffic

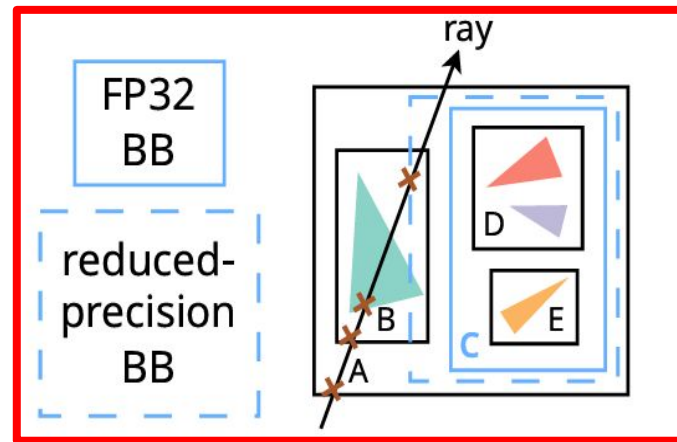
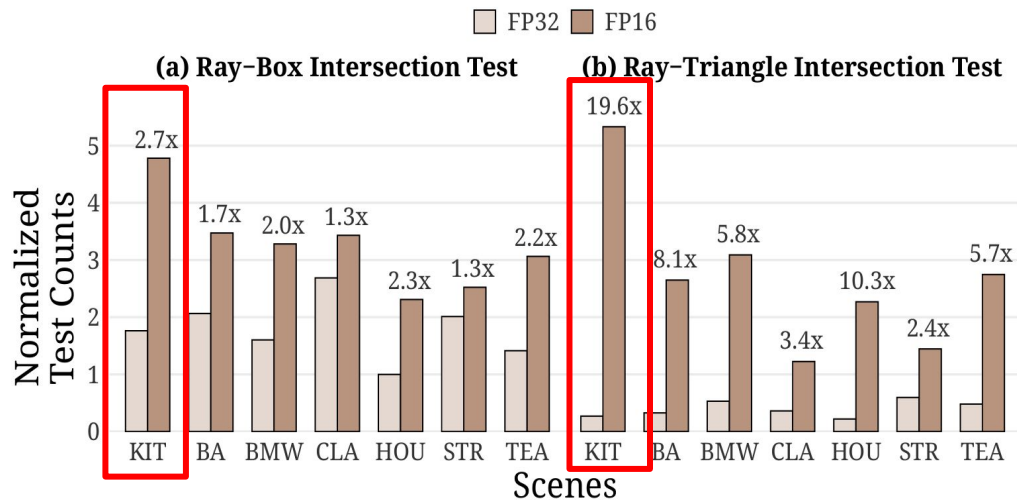
- **Problem of BVH**: Bounding Box (BB) dominates memory traffic:
- → **75%** of **DRAM accesses** (**memory-bound after HW acc.**)





Compressed BVH

- Compressed BVH **in low-bit formats** reduced memory traffic
- Problem** : **Directly use FP16** cause **quantization error**
(change the size of BBs) → increase BVH tree traversal steps.





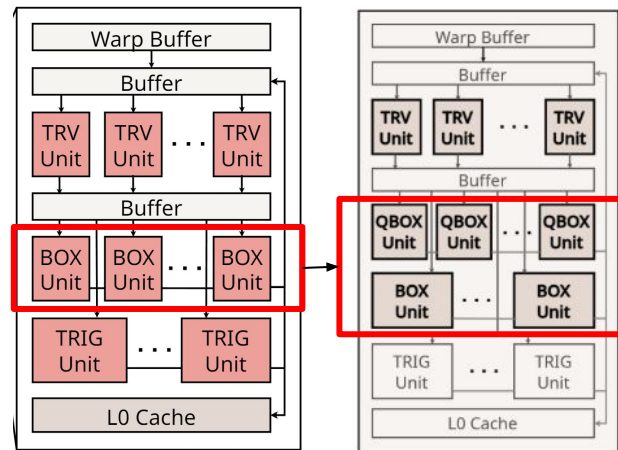
Limitations of RT accelerator

- Modern RT accelerator employs FP32 BOX units for ray-box intersection test. (FP32 is power hungry over INT8)
- Proposed: AQB8 – Accelerator for Quantized BVH with 8-bit Traversal.**

Energy Per Operation of BOX Unit (TSMC 40nm)

FP32	$1.38e-1 \text{ nJ}$
INT8	$2.43e-2 \text{ nJ}$

$5.67x$





What's new in AQB8?

- AQB8 is an energy-efficient RT accelerator that realizes the ray-box intersection test using integer arithmetic.
- **Performance Gains (vs. baseline):**
 - **70%** reduction in memory traffic → (mix-precision BVH)
 - **49%** lower energy consumption → (low-bit data and arithmetics)
 - **27%** smaller hardware area → (increasing INT8 functional units)
 - **1.82×** speedup → (geometric mean across realistic RT workloads)



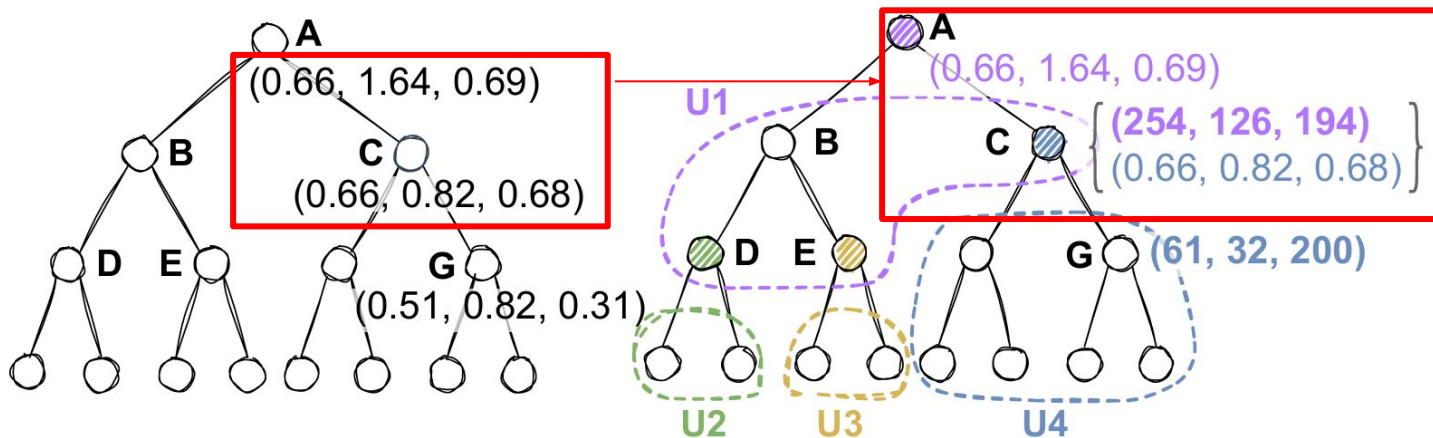
Challenges of AQB8 RT Accelerator

- How to **quantize BB** coordinates without significantly increase ray tracing performance overhead?
- How to create **a mix-precision BVH tree** with the appropriate number of quantized and reference BBs?
- How to **replace FP32 BOX unit with INT8 QBOX unit?**



AQB8 Multilevel Quantization

- Divide mixed-precision BVH into BB **clusters (U1-U4)**, each BB cluster includes:
 - A reference BB as FP32 format.**
 - Quantized INT8 BBs based on reference BB coordinates.**





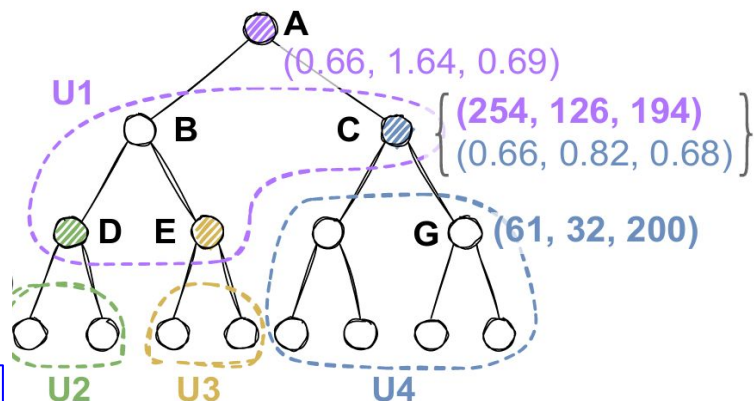
Quantized BVH Tree Construction

- **Problem**: How to decide the size of each BB cluster?
- **Method**: Modify **Surface Area Heuristic (SAH)** cost function.

$$C(N) = \frac{1}{S(N)} \left[\sum_{N_i} T(N_i) S(N_i) + c_i \sum_{N_l} S(N_l) |N_l| \right]$$

$$T(N_i) = \begin{cases} c_t + c_s, & \text{if } \pi(N_i) = \text{SWITCH}, \\ c_t, & \text{if } \pi(N_i) = \text{STAY}. \end{cases}$$

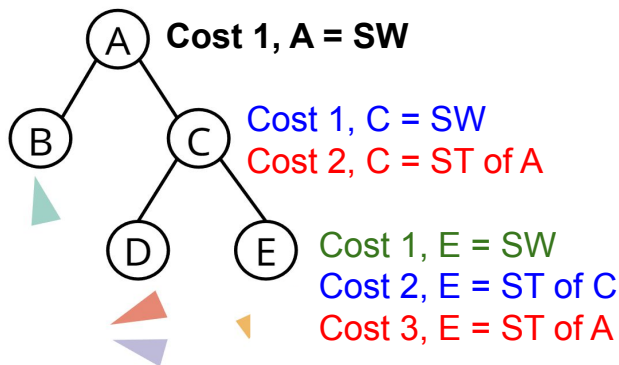
- **SWITCH (SW)**: As the reference BB in a new cluster
- **STAY (ST)**: Quantized node in BVH tree





Cost Function of Mixed-precision BVH

- **Root node** must be the SWITCH state.
- **Each node** can be the **SWITCH** state or the **STAY** state.
- AQB8 uses **dynamic programming (DP)** { $O(n(\log n)^2)$ } instead of $O(2^n)$ to decide the state of each BVH node with the small cost ($C(N)$)



$$C(N) = \frac{1}{S(N)} \left[\sum_{N_i} T(N_i) S(N_i) + c_i \sum_{N_l} S(N_l) |N_l| \right]$$
$$T(N_i) = \begin{cases} c_t + c_s, & \text{if } \pi(N_i) = \text{SWITCH}, \\ c_t, & \text{if } \pi(N_i) = \text{STAY}. \end{cases}$$



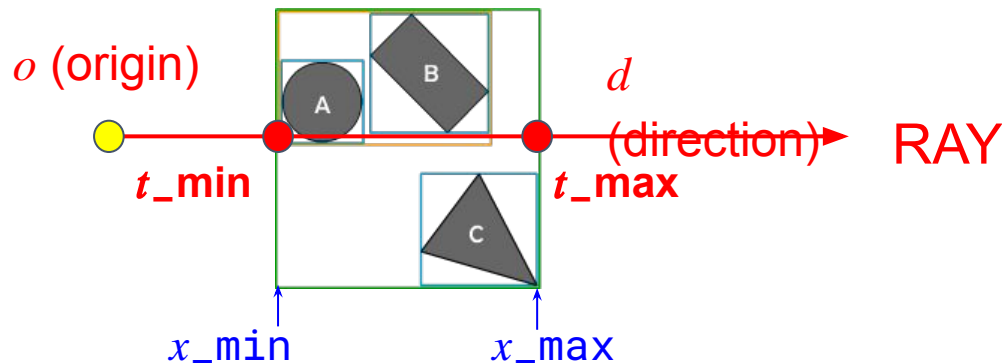
Quantized Ray-Box Intersection

- **Problem:**
 - Low-bit format Ray-Box intersection tests **without decompress (INT8 $\rightarrow$ FP32) BB.**
- **Method:**
 - Instead of decompressing BBs, AQB8 **quantizes rays to match cluster-local INT8 BBs.**
- Uses scaling factors:
 - $\underline{S_w} = 2^{-7}$
 - $\underline{S_x} = \underline{\text{scaling factor}}$ of the reference BB (per cluster)



Quantized Ray-Box Intersection

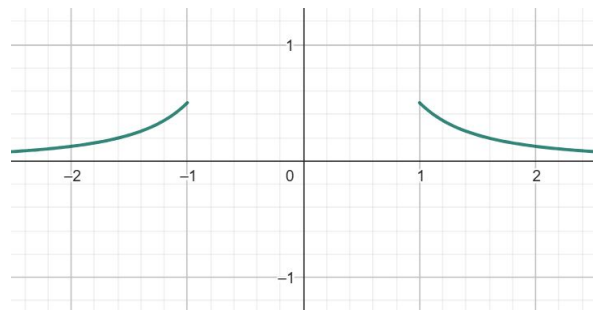
- Ray-box intersection test formula: $t = (x - o) / d$
- Definition: $t = wx + b$, $w = (1 / d)$, $b = -o / d$
 - x is the quantized BB coordinates relative to reference BB.
 - w is the constant, while b varies across different reference BBs.





Low-Bits Representation of qw

- We represent qw using a low-bits **floating point**. ($w = 1 / d$)
- d is the ray direction unit vector.
- $f(w)$ is the probability density function of w :
 - $d \in [-1, 1]$; $w \in (-\infty, -1] \cup [1, \infty)$



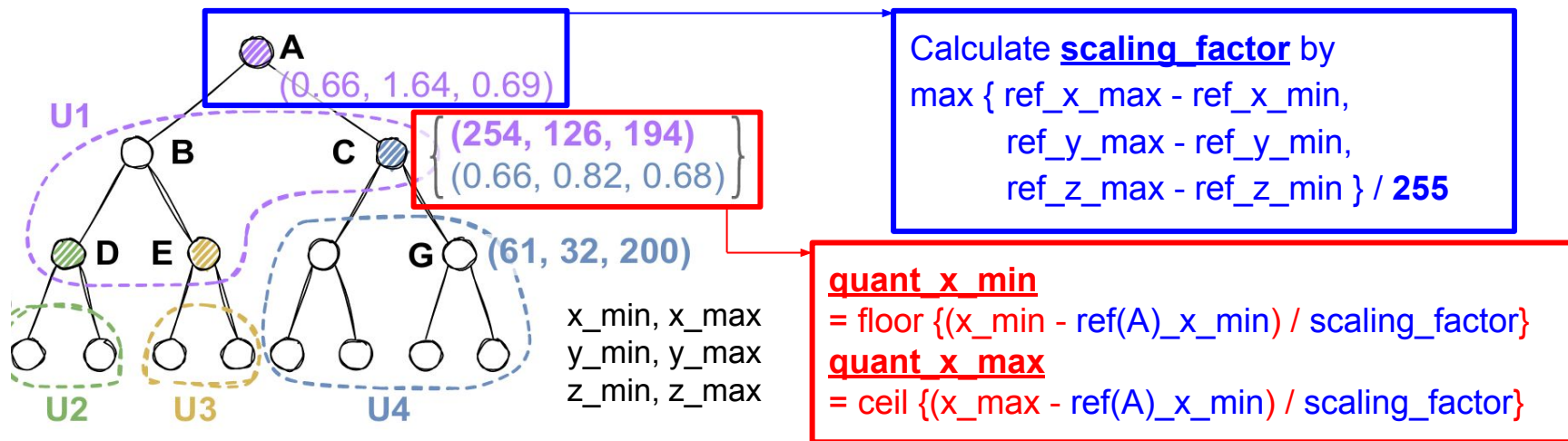
$$f(w) = \begin{cases} \frac{1}{2w^2}, & \text{if } |w| \geq 1, \\ 0, & \text{otherwise.} \end{cases}$$

Most of w values are close to **-1 and 1**.
→ **Narrow down the # of bits to represent w .**



Bounding Box Quantization

- Directly use low-bit format cause quantization error
 - → **Quantize BB coordinrates derives from a reference BB belonging to the same BB cluster.**
- Each **quantized BB** uses **six INT8 ([0, 255]) values**.





Quantized Ray-Box Intersection

- **Problem**: How to quantize FP32 w , x , and b to INT arithmetic?

Ray-box intersection test formula: $t = wx + b$

- $S_w = 2^{-7}$
- $S_x = \text{scaling_factor}$

- $q_w = w / S_w$
- $q_x = x / S_x$
- $q_t = t / (S_w S_x)$
- $q_b = b / (S_w S_x)$

$$S_w S_x q_t = S_w q_w S_x q_x + S_w S_x q_b$$

Diagram illustrating the quantization of the ray-box intersection test formula. The variables q_w , q_x , and q_b are shown in boxes, indicating they are quantized values. The scaling factors S_w and S_x are also shown. The result q_t is shown in a box, indicating it is a quantized value. The diagram also shows the quantization of the original variables w , x , and b to INT arithmetic.

Labels below the equation:

- $S_w S_x q_t$: INT32
- $S_w q_w$: INT8 (quantized BB coordinate)
- $S_x q_x$: INT32
- $S_w S_x q_b$: INT32

FP14

$$S_w S_x q_t = S_w q_w S_x q_x + S_w S_x q_b$$

Quantize FP32 w , x , and b to INT arithmetic.



Convert q_w from FP32 to FP14

	Sign 1 bit	Exponent 8 bit	Mantissa 23 bit
FP32 format value =	$(-1)^{\underline{S}}$	$2^{\underline{(\text{Exp} - 127)}}$	$1.\underline{M} (\underline{1000110}...)$
Custom FP14 format value =	Sign 1 bit $(-1)^{\underline{S}}$	Exponent 5 bit $2^{\underline{(\text{Exp})}}$	INT8 Constant 8 bit $\underline{C} (\underline{1}\underline{1000110})$
			$\div S_w (2^{-7})$



Convert q_w from FP32 to FP14

Ex: $w = 2$ in FP32 Format
 $= (-1)^{\underline{0}} \times 2^{\underline{(128-127)}} \times \underline{1}$

$\div S_w (2^{-7})$

unbiased

q_w in FP14 format
 $= (-1)^{\underline{0}} \times 2^{\underline{(1)}} \times \underline{256}$

Sign

1 bit

0

Exponent

8 bit

1 0 0 0 0 0 0 0

1 0 0 0 0 1 1 1

- 127 (exponent bias)

0 0 0 0 1 0 0 0

shift for 8 bit constant

0 0 0 0 1

5 bit

Mantissa

23 bit

0 0 0 0 0 0 0 0 ... 0

0 0 0 0 0 0 0 0 ... 0

1.0000000 → 10000000.

8 bit Constant

1 0 0 0 0 0 0 0



Quantized Ray-Box Intersection

- Ray-box intersection test: $t = wx + b$

$$S_w S_x q_t = S_w q_w S_x q_x + S_w S_x q_b$$

sign	exponent	mantissa
1	5	8

Format q_w as **FP14**

- 1 sign bit = i_w
- 5 exponent bits = r_w
range of $r_w = [0 - 31]$
- 8 mantissa bits = m_w

$$q_t = i_w 2^{r_w} m_w q_x + q_b$$

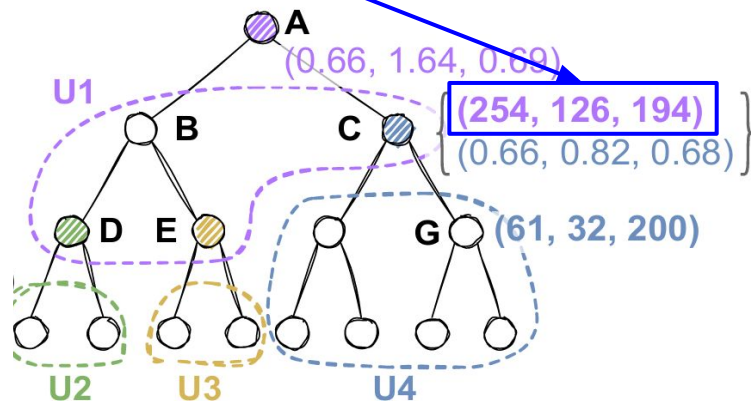
INT8

left shift

2's complement

INT32

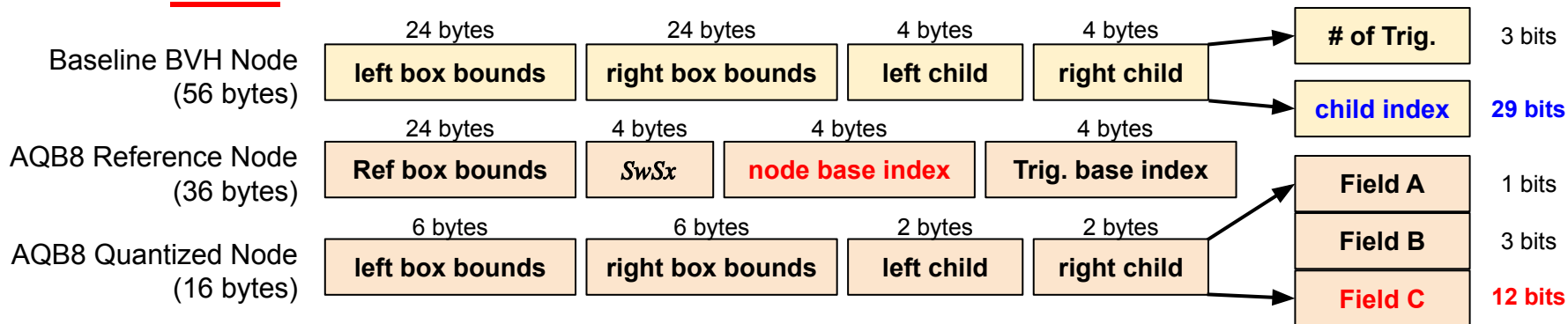
coordinate of quantized BB





AQB8 Memory Layout Optimizations

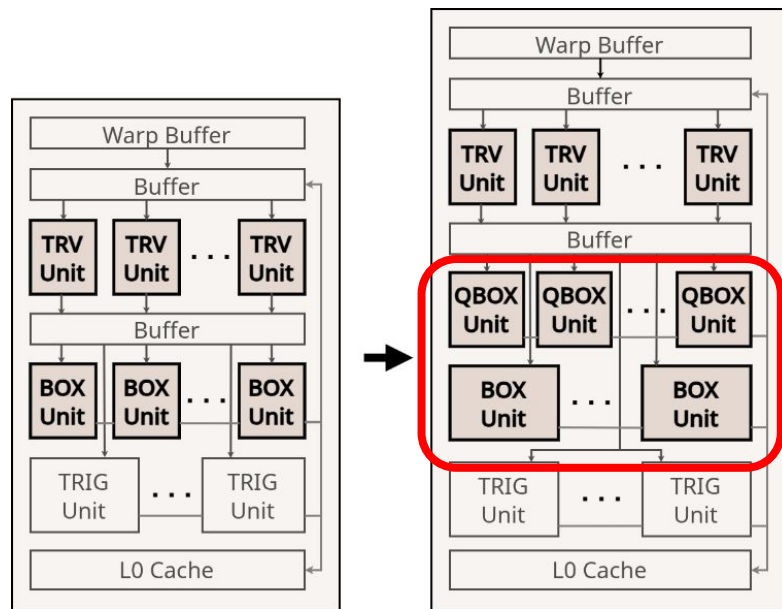
- BB consists of **six INT8 (= 6 bytes)** coordinates.
- The field of child node records the **node index** of a BVH tree.
- AQB8 **reduces child node field size** by using the **base index** of the reference node and the **offset** of the quantized nodes.
- The maximum number of quantized nodes in a cluster is **$2^{\text{Field C bit}}$** .





AQB8 RT Accelerator

- Increasing the number of INT functional units in the RT accelerator
- Modified TRV Units:
 - Add **SWITCH, STAY, LEAF** logic.
- Create QBOX Units:
 - New units for **quantized ray-box intersection**.
 - Use **low-bit integer arithmetic**.
- Modified BOX Units:
 - Add additional ray quantization logic and reference BB data retrieving.



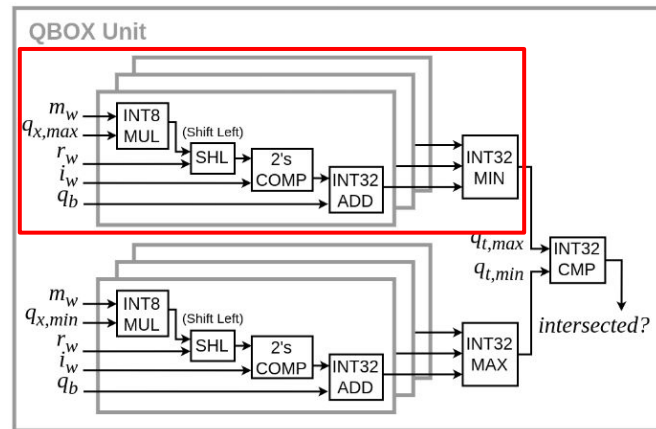
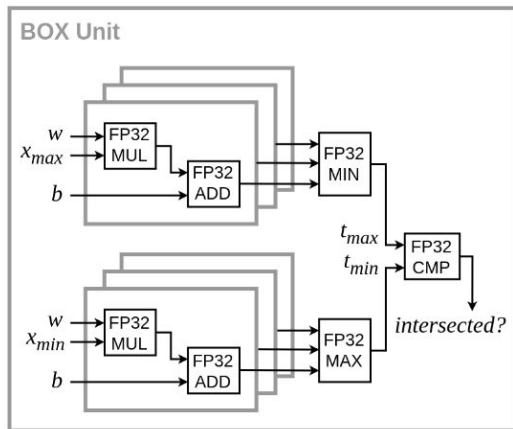


Quantized RT Accelerator

- QBOX Units:
- New units for **quantized ray-box intersection**.
- $qt = qw \cdot qx + qb$

$$q_t = \underbrace{i_w 2^{r_w}}_{\text{left shift}} \underbrace{m_w q_x}_{\text{INT8}} + q_b$$

INT32





Environmental Setups

- Implement AQB8 on cycle-accurate **Vulkan-Sim**.
- Hardware synthesis using **TSMC 40nm**.
- Evaluate **2-wide, 6-wide, and compressed BVH**.

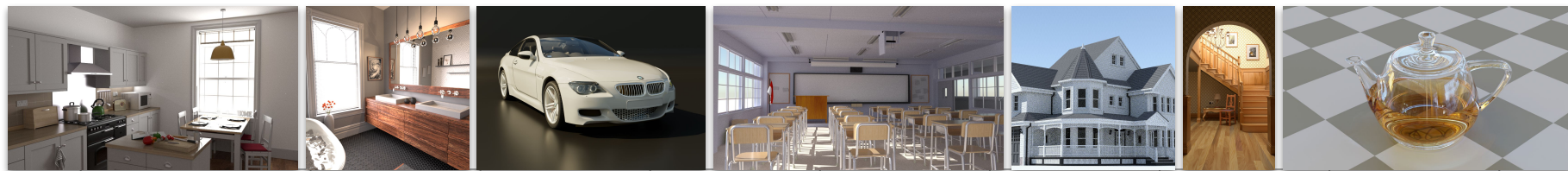
Number of SMs	30
Max Warps / SM	32
Warp Size	32
Warp Scheduler	GTO
# of Registers / SM	32768
Global Memory Clock	3500MHz

L1 Data Cache / SM	64KB
L2 Unified Cache	3MB
Compute Core Clock	1365MHz
# of RT Accelerators / SM	1
RT Accelerator Warp Buffer Size	4
RT Dedicated L0 Cache	8KB



Evaluated Scene Workloads

- Scenes are used to generate 256 x 256 pictures.
- **AQB8 yields a small number of reference (ref.) BBs** because a ref. BB can have multiple quantized BBs in a cluster

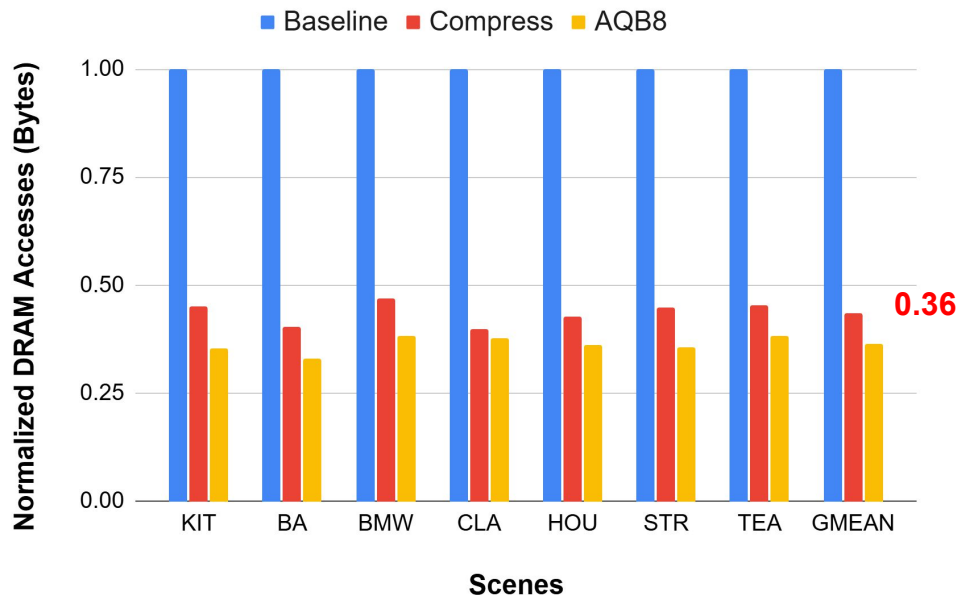


Scenes	Kitchen	Bathroom	BMW	Classroom	House	Staircase	Teapot
# of triangle	1.44M	0.59M	0.79M	0.10M	1.82M	0.26M	0.13M
# of Quant-BB	1.39M	0.55M	0.78M	0.09M	1.79M	0.23M	0.11M
# of Ref. BB	0.021M	0.005M	0.012M	0.003M	0.032M	0.003M	0.001M



Memory Traffic Reduction

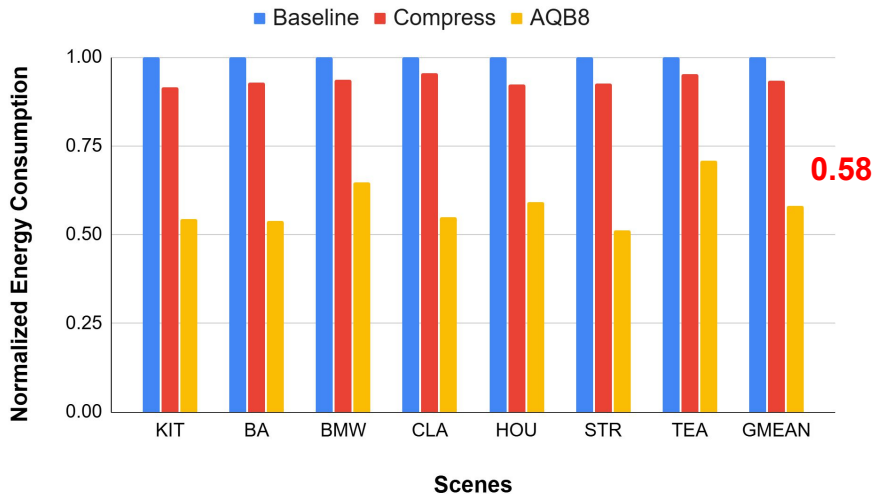
- **DRAM Accesses:** **61–76%** reduction (avg. 64%).
- BBs are stored in a substantially smaller size (**FP32 to INT8**).





Energy Consumption

- **AQB8** reduces **total energy by 29–49%** (avg. 42%)
- Most ray-box intersections are handled with **low-bit integer arithmetic**, reducing dynamic power in RT accelerator.

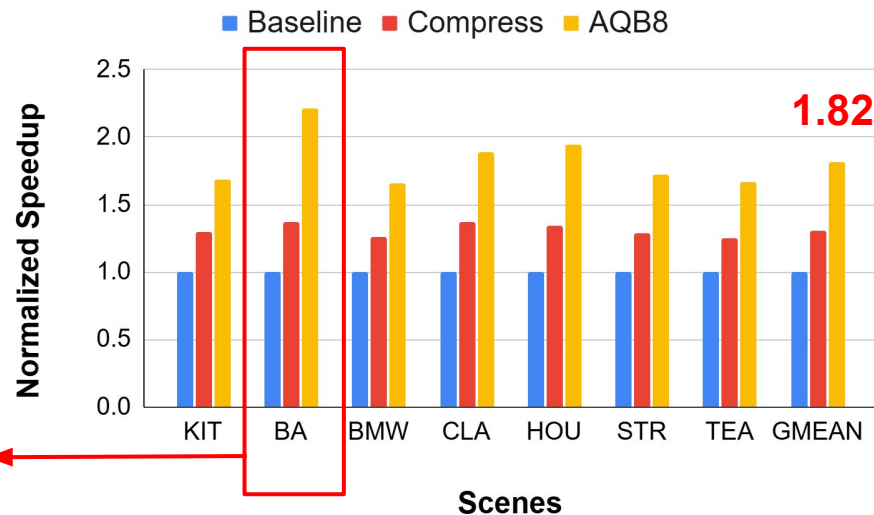




Performance

- Speedup up to **1.82× on 2-wide BVH tree.**
- We use INT8 instead of FP32 to represent BB:
 - Reduces computation overhead.
 - Lowers memory transfer time.

AQB8 achieves **better performance** when handling scenes with more **scattered triangles**.





AQB8 GitHub

Conclusion

- **AQB8** is a domain-specific **Ray Tracing (RT) accelerator** designed to overcome memory and compute bottlenecks of RT
- **Multi-level quantization on BVH**
 - reduce memory traffic over 70%.
- **Mixed-precision BVH tree**
 - reduce quantization error and only yields less than 10% ray-box intersection overhead.
- **Integrating QBOX hardware units in the RT accelerator**
 - reduce hardware complexity of FP32 QBOX unit and achieves 1.82X speedup



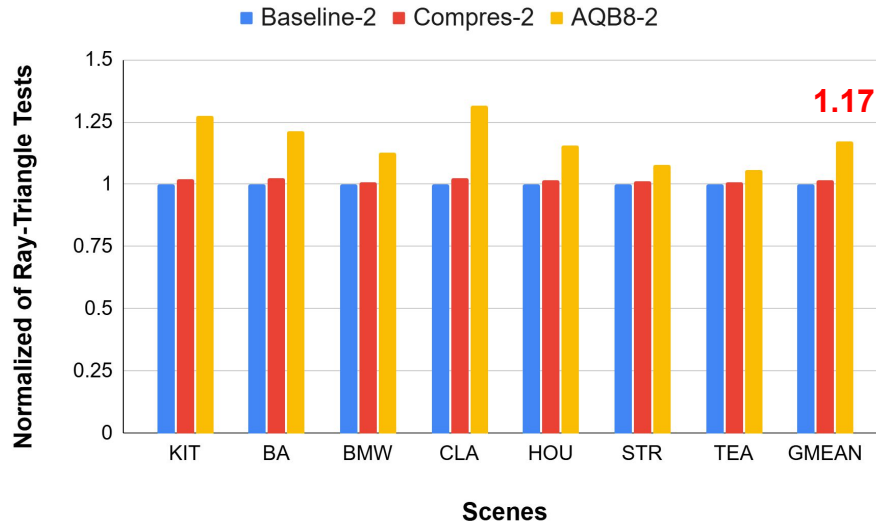
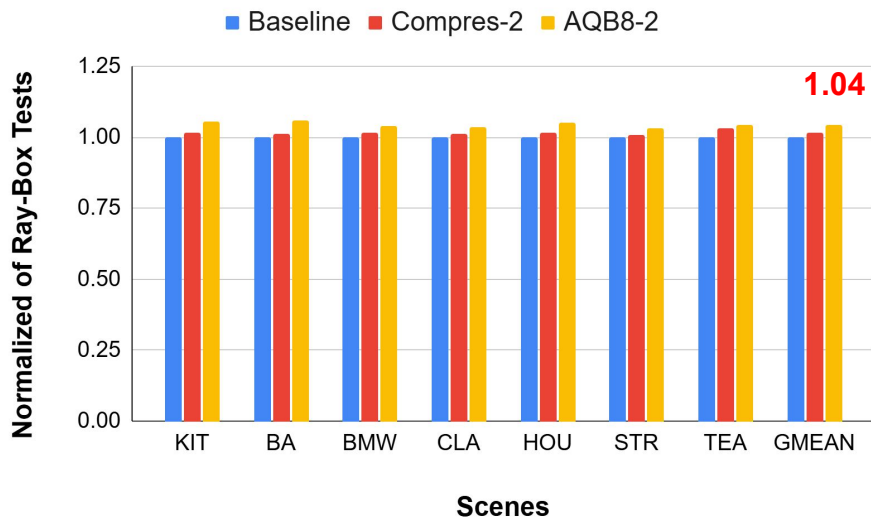
Thanks for Listening



Tree Traversal Steps

Slight overhead due to **quantization error**:

- Ray-box tests: **+3–6%** / Ray-triangle tests: **+6–31%**





Cost Function of Mixed-precision BVH

- Parameters: $[ct, ci, cs] = [0.5, 1, 1]$ (empirically decided)
- ct the cost of a **traversal step**.
- ci the cost of **ray-triangle intersection**.
- cs the cost of **switching clusters**.
 - If cs increases, **the number of clusters will decrease**.
- The above ct, ci, cs values can apply on different scenes.

$$T(N_i) = \begin{cases} c_t + c_s, & \text{if } \pi(N_i) = \text{SWITCH}, \\ c_t, & \text{if } \pi(N_i) = \text{STAY}. \end{cases}$$

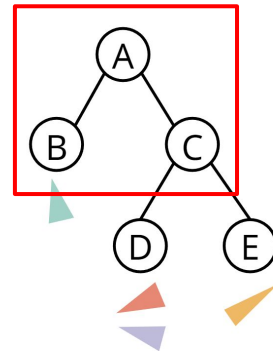
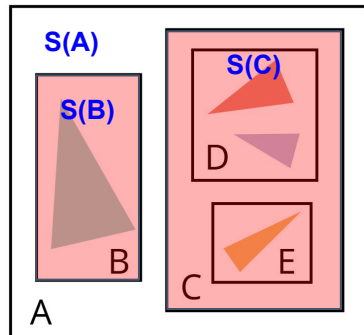


Clustering Cost Function

The cost function is based on Surface Area Heuristic (SAH):

- $C(N)$ is the cost of the BVH tree traversal.
- $S(N)$ is the surface area of node N .
- $S(N_i)$ is the surface area of the internal node.
- $S(N_l)$ is the surface area of the leaf node.
- Ex: the $S(N_i)$ and $S(N_l)$ of $S(A)$ are $S(C)$ and $S(B)$.

$$C(N) = \frac{1}{S(N)} \left[c_t \sum_{N_i} S(N_i) + c_l \sum_{N_l} S(N_l) |N_l| \right]$$





Hardware Area

		TRV	QBOX	BOX	TRIG
Hardware Area (mm^2)	Baseline-2	7.88	-	82.13	192.92
	Compress-2	8.12	-	90.58	192.92
	AQB8-2	9.80	16.08	97.05	192.92

- **QBOX unit (INT8) in AQB8:**
 - **5.1× smaller** due to **replacing FP32 hardware with INT8**.
- **TRV and BOX units in AQB8:**
 - Slightly larger due to **cluster** (reference BVH node) logic.

Baseline RT Core: 64 TRV, 51 BOX, 11 TRIG

→ $64 \times 7.88 + 51 \times 82.13 + 11 \times 192.92$

→ **6815.07**

AQB8 Core: 78 TRV, 53 QBOX, 9 BOX, 13 TRIG

→ $78 \times 9.80 + 53 \times 16.08 + 9 \times 97.05 + 13 \times 192.92$

→ **4998.05 (27% reduction)**



Performance

- Speedup up to **1.43x** on **6-wide BVH tree**.

